

# SWE3025 Homework 2

## Computer Security

Hyungjoon Koo

Sungkyunkwan University

# Instructions

- Homework assignment #2
  - 3 questions: Q1 (20 pts), Q2 (30 pts), Q3 (50 pts)
  - Deadline: midnight (11:59 PM) on May 29, (Friday), 2026
  - Ethics: Copying, publishing, or sharing your code/solution is NOT allowed
  - You may consult LLMs for this assignment.
- Submission (via iCampus only)
  - Must be submitted on an individual basis
  - Submit a single zipped file; the file name must be your student ID with the .zip extension
  - The submission must contain: a PDF report, three Python scripts (1\_exploit.py for Q1, 2\_exploit.py for Q2, and 3\_exploit.py for Q3), a single demo video clip for all questions.
- Evaluation
  - The scripts must run with python3.7 or higher.
  - The environmental setup must be done via Docker for all questions
  - You will receive full credit only if your solution follows each vulnerability description.
  - No point for late submissions

**Docker** is an open-source platform for deploying and managing applications through containerization. A Docker container packages software into a standalone unit—including code, runtime, system tools, and libraries—replicating a complete system environment, guaranteeing the same environment for everyone.

## Why Docker for this homework?

All questions must be configured and executed within Docker containers for fair evaluation. You are **not** allowed to modify the default Docker settings.

# Docker Installation

## On Ubuntu in Windows WSL:

```
1 # Update apt packages
2 $ sudo apt update -y && sudo apt upgrade -y
3 $ sudo apt install -y apt-transport-https ca-certificates curl software-properties-common
4 # Add Docker GPG key and repository
5 $ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
6 $ echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release
   -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
7 $ sudo apt update -y && sudo apt install -y docker-ce docker-ce-cli containerd.io
8 $ sudo usermod -aG docker <USER>
9 $ sudo service docker start && docker run hello-world
```

## On Ubuntu (simple):

```
1 $ curl -fsSL https://get.docker.com/ | sudo sh
2 $ sudo usermod -aG docker <USER>
3 $ docker run hello-world
```

# Building a Docker Image

With the provided Dockerfile and file directory, build and run your own image as follows. You **must** use the `--privileged` option when running the container.

```
1 # Build the Docker image
2 $ cd <Dockerfile path>
3 $ docker build . -t <tag_name>
4
5 # Run in privileged mode
6 $ docker run -it --privileged <tag_name> /bin/bash
7 user@4d7115f58a8e:~$ ls -l
8 [+] ASLR enabled: 2 (full)
9 user@844b25ce87fc:~$ ls -l
10 total 20
11 drwxr-xr-x 1 root root 4096 May 11 02:34 1.code_injection
12 drwxr-xr-x 1 root root 4096 May 11 02:34 2.ret2libc
13 drwxr-xr-x 1 root root 4096 May 11 02:34 3.rop
14 -rwxr-xr-x 1 root root 649 May 11 02:30 aslr_on.sh
15 drwxr-xr-x 1 root root 4096 May 6 02:19 peda
```

Refer to the official Docker documentation for further guidance. Once the image is set up, you will have three tasks to complete inside your Docker instance.

## Caution

Files created inside the Docker container are deleted once you exit. Be sure to back them up beforehand.

# ASLR Configuration

This assignment requires **ASLR enabled** at runtime.

The Docker image is configured to turn ASLR on automatically when the container launches.

```
1 [+] ASLR enabled: 2 (full)
```

If the message does not appear, verify the kernel setting *inside the container*; the value must be 2:

```
1 $ cat /proc/sys/kernel/randomize_va_space
2 2
```

## ASLR Troubleshooting

ASLR inside a Docker container inherits from the host kernel.

If the value above has not set to 2, you may enable it **on the host** (not inside the container) before re-launching:

```
$ sudo sh -c 'echo 2 > /proc/sys/kernel/randomize_va_space'
```

Please make sure that ASLR is **turned on**.

# Library offset

For ret-to-libc / ROP, you need the **offset** of a libc symbol (e.g. `system`) relative to the libc base. Combined with a runtime leak of any libc address, this gives you the absolute address at exploit time.

**Step 1.** Find which libc the binary loads with `ldd`:

```
1 $ ldd ./retlib
2   linux-gate.so.1 (0xf7fc3000)
3   libc.so.6 => /lib/i386-linux-gnu/libc.so.6 (0xf7d82000)
4   /lib/ld-linux.so.2 (0xf7fc5000)
```

The path on the right is the exact libc your exploit must use — offsets differ across distros and versions.

**Step 2.** Look up symbol offsets inside that libc with `objdump`:

```
1 $ objdump -d /lib/i386-linux-gnu/libc.so.6 | grep "system"
2 00044980 <__libc_system@@GLIBC_PRIVATE>:
```

# Problems

- Q1. Code injection attack
- Q2. Return-to-libc attack
- Q3. Return-oriented programming (ROP) attack

## Note

We provide a pre-built binary for each question when building a Docker image. Use the given binary instead of **modifying** the source and **recompile** it.



## Q1: Code Injection Attack (20 points)

Your task in Q1 is to spawn a shell by **injecting shellcode** into a vulnerable process's memory and executing it.

With a careful look at the `stack.c` source (in `1.code_injection`) write `1_exploit.py` to spawn a shell by hijacking control flow.

- The program prints the runtime address of `buffer[]`; you *must* use this address rather than any hard-coded gdb value.
- Verify success by running basic commands (`ls`, `id`) inside the spawned shell.
- Submit a report describing how your exploitation works.

### Compilation flags:

```
gcc -z execstack -fno-stack-protector -m32 -no-pie -g -o stack stack.c
```

## Q1: Environment

### Environment (use checksec)

|        |                        |
|--------|------------------------|
| Arch   | 32-bit (i386)          |
| Canary | None                   |
| NX     | Off (executable stack) |
| RELRO  | Partial                |
| PIE    | None                   |
| ASLR   | Enabled at runtime     |

## Q1: Expected Output

An example successful run of `1_exploit.py` should spawn an interactive shell:

Listing 1: Running `stack` with an injected shellcode.

```
1 user@4729d3ceb14d:~/1.code_injection$ python3 1_exploit.py
2 [+] Starting local process './stack': pid 270
3 [*] Switching to interactive mode
4 $ ls
5 1_exploit.py  stack  stack.c
6 $ id
7 uid=1000(user) gid=1000(user) groups=1000(user)
```

## Q2: Return-to-libc (30 points)

Q2 aims to spawn a shell by **returning into a libc function** (e.g., `system()`).

With a careful look at the `retlib.c` source, write `2_exploit.py` to spawn a shell via ret-to-libc.

- Every run yields a different libc base because of ASLR, so any hard-coded libc base address will not work.
- You can spawn a shell with `system("/bin/sh")`
- Submit a report describing how your exploitation works.

### Compilation flags:

```
gcc -fno-stack-protector -z noexecstack -m32 -no-pie -g -o retlib retlib.c
```

## Q2: Environment

### Environment (use checksec)

|        |                    |
|--------|--------------------|
| Arch   | 32-bit (i386)      |
| Canary | None               |
| NX     | On                 |
| RELRO  | Partial            |
| PIE    | None               |
| ASLR   | Enabled at runtime |

## Q2: Expected Output

A successful run of `2_exploit.py` computes the `libc` base from the leaked `stdout` pointer and spawns an interactive shell.

Listing 2: Running `retlib` with a `ret2libc` payload.

```
1 user@4729d3ceb14d:~/2.ret2libc $ python3 2_exploit.py
2 [+] Starting local process './retlib': pid 312
3 [+] stdout @ 0xf7fabd40
4 [+] libc @ 0xf7d82000
5 [+] system @ 0xf7dd18e0
6 [+] /bin/sh @ 0x804a008
7 [*] Switching to interactive mode
8 $ id
9 uid=1000(user) gid=1000(user) groups=1000(user)
10 $ ls
11 2_exploit.py  retlib  retlib.c
```

## Q3: ROP (50 points)

The goal of Q3 is to spawn a shell against a binary protected by **ASLR + PIE + NX** simultaneously. No address is known up front — the exploit must construct every gadget address at runtime.

With a careful look at the `rop.c` source, write `3_exploit.py` to spawn a shell via a ROP chain.

- You should bypass PIE by using gadgets.
- The ROPgadget tool is already installed, so you can use it to search gadgets in a given binary.
- Sometimes you may need a ret gadget for stack alignment.
- Submit a report describing how your exploitation works.

### Compilation flags:

```
gcc -fno-stack-protector -fcf-protection=none -z noexecstack -pie -fpie -g -o rop  
rop.c
```

## Q3: Environment

### Environment (use checksec)

|        |                    |
|--------|--------------------|
| Arch   | 64-bit (x86-64)    |
| Canary | None               |
| NX     | On                 |
| RELRO  | Full               |
| PIE    | On                 |
| ASLR   | Enabled at runtime |



## Q3: Expected Output

A successful run of `3_exploit.py` performs two leaks (PIE + libc), builds a ROP chain, and spawns an interactive shell.

Listing 3: Running rop with a exploit payload.

```
1 user@4729d3ceb14d:~/3.rop$ python3 3_exploit.py
2 [+] Starting local process './rop': pid 358
3 [+] PIE base = 0x55e7c1a5b000
4 [+] libc base = 0x7f3abe200000
5 [+] /bin/sh = 0x55e7c1a5d0bb
6 [*] Switching to interactive mode
7 $ id
8 uid=1000(user) gid=1000(user) groups=1000(user)
9 $ ls
10 3_exploit.py  rop  rop.c
```